

FINAL LAB PROJECT REPORT

Omni

A Digital Health & Telemedicine Companion for Android

Team Members: 【Member 1 name】 , 【Member 2 name】 , 【Member 3 name】

Submitted To: 【Instructor / Supervisor name】

Course: 【Course title and code】

Department: 【Department / Program】

Institution: 【University / College name】

Date: September 2026

Table of Contents

1 Abstract.....	1
2 Introduction.....	2
2.1 Background and Motivation.....	2
2.2 Problem Statement and Objectives.....	2
2.3 Scope of the Project.....	3
3 Literature Review and Related Work.....	3
3.1 Overview of Similar Applications.....	3
3.2 Key Technologies and Frameworks.....	4
4 Methodology.....	4
4.1 Development Approach.....	5
4.2 Tools and Technologies.....	5
4.3 Project Timeline and Milestones.....	5
5 System Design and Architecture.....	6
5.1 High-Level Architecture.....	6
5.2 Core Components.....	7
5.3 Data Model.....	7
5.4 UI/UX Design Considerations.....	7
6 Implementation.....	8
6.1 Development Process.....	8
6.2 Key Features and Functionalities.....	8
6.3 AI Meal-Analysis Pipeline.....	9
6.4 Code Structure and Major Modules.....	9
7 Results and Discussion.....	9
7.1 Project Achievements.....	9
7.2 Challenges Faced and Solutions.....	10
7.3 Comparison with Initial Objectives.....	11

8 Conclusion and Future Work.....	11
8.1 Summary of Findings.....	11
8.2 Project Limitations.....	12
8.3 Future Work.....	12

Note: This Table of Contents is generated via field codes. To refresh the page numbers after editing, right-click the table and choose "Update Field".

1 Abstract

Omni is a native Android health-and-wellness application that consolidates preventive self-care, emergency assistance, nutrition intelligence, sleep and activity tracking, peer social interaction, and doctor-to-patient telemedicine into a single, cohesive mobile product. The project was undertaken as the team’s final laboratory project and was built entirely in Kotlin using Jetpack Compose for a fully declarative user interface, Firebase for authentication and real-time data, and a purpose-built Node.js/TypeScript backend that brokers all privileged operations, including artificial-intelligence (AI) inference.

The primary objective was to design and implement a production-grade, multi-feature health application that keeps sensitive logic and secret keys off the client while still delivering a fast, offline-tolerant user experience. The methodology followed an iterative, phase-based development model in which the system was delivered across sixteen numbered phases, each ending in a device-verified increment. The application layers a Model-View-ViewModel (MVVM) presentation architecture over a repository data layer, and defers all AI, email, verification, and entitlement operations to the server tier where they can be validated and secured.

The completed system comprises roughly 40,000 lines of Kotlin across 151 source files and approximately thirty screens, backed by a fifteen-file TypeScript service. Key outcomes include an AI meal-analysis pipeline powered by Google Gemini, a subscription tier (“Omni+”) integrated through RevenueCat with server-side entitlement granting, a verified-doctor consultation channel, an offline-tolerant step counter, and a MapLibre-based emergency (SOS) map. The report documents the motivation, design, implementation, and evaluation of the system, and concludes with a candid discussion of limitations and a roadmap for future work.

2 Introduction

2.1 Background and Motivation

Personal health management is fragmented across many single-purpose applications: one app tracks steps, another logs meals, a third handles messaging with a clinician, and yet another provides first-aid guidance. This fragmentation forces users to maintain several accounts, learn several interfaces, and manually reconcile data that is conceptually related. The motivation behind Omni was to demonstrate that these concerns can be unified in one well-architected Android application without sacrificing security or responsiveness.

The team was additionally motivated by the engineering challenge of doing this correctly. Health applications handle sensitive personal data and, in the case of AI features and paid subscriptions, must protect API secrets and validate entitlements. A naive implementation that embeds AI keys in the mobile client or trusts the client to report its own subscription status would be insecure. Omni was therefore designed from the outset around a trusted backend that mediates every privileged action.

2.2 Problem Statement and Objectives

The problem this project addresses is the absence of a single, secure, and pleasant Android application that combines everyday wellness tracking with on-demand access to professional and emergency help. To address it, the project pursued the following objectives:

- **Unified experience** — deliver a unified Android client covering activity, sleep, nutrition, first aid, social interaction, and telemedicine;
- **Security by design** — keep all secrets and privileged logic server-side, so the client never holds an AI key or decides its own subscription status;
- **Useful intelligence** — provide AI-assisted nutrition analysis that returns structured, validated results rather than free-form text;
- **Sustainable model** — support a paid tier with correct, tamper-resistant entitlement handling; and
- **Design fidelity** — build the interface as a faithful, pixel-checked realization of a Figma design system.

2.3 Scope of the Project

The scope covers a complete Android application and its supporting cloud backend. In scope are the mobile client (all screens, view models, and repositories), the Firebase project (authentication, Firestore data, and cloud messaging), the Node.js/TypeScript backend (one-time-password email verification, AI meal analysis, doctor verification, messaging, and subscription entitlement), and the automated test suites for both tiers. Out of scope are a public Play Store release (the application currently uses a development package identifier and is distributed by sideloading) and any native iOS client; the project targets Android exclusively.

3 Literature Review and Related Work

3.1 Overview of Similar Applications

Several established products informed the design of Omni. General wellness platforms such as Google Fit and Samsung Health aggregate step counts, sleep, and activity, and served as

reference points for the tracking features. Nutrition applications such as MyFitnessPal popularized meal logging with nutritional breakdowns; Omni differs by using a large language model to estimate nutrition from a natural-language meal description rather than requiring the user to search a food database. Telemedicine services demonstrate patient-to-clinician messaging and consultation booking, which Omni reproduces through its verified-doctor channel. Finally, dedicated first-aid applications from organizations such as the Red Cross validated the value of concise, illustrated emergency guidance, which Omni delivers through animated step-by-step guides.

The distinguishing contribution of Omni relative to these products is consolidation with a security-first backend: rather than specializing in one vertical, it combines tracking, nutrition intelligence, social features, telemedicine, and emergency assistance while ensuring that AI inference and subscription entitlement are never trusted to the client.

3.2 Key Technologies and Frameworks

The project rests on a modern Android and cloud technology stack. On the client, Jetpack Compose provides a declarative UI toolkit that replaces the legacy XML view system with composable functions and unidirectional state flow; Material 3 supplies the visual language. Kotlin coroutines and Flow provide structured concurrency for asynchronous work. On the data side, Firebase Authentication, Cloud Firestore, and Cloud Messaging provide identity, a real-time NoSQL document store, and push notifications respectively. Coil renders images and animated GIFs, and RevenueCat abstracts subscription billing.

On the server, Node.js with the Express framework and the Firebase Admin SDK host the privileged endpoints; the Google Gemini generative-language API performs nutrition inference; and SendGrid delivers verification email. The following table summarizes the principal technologies and the role each plays in the system.

Table 1 Core technologies and their roles

Layer	Technology	Role in Omni
Language	Kotlin 2.4 / TypeScript	Client written in Kotlin; backend and Cloud Functions in TypeScript
UI	Jetpack Compose + Material 3	Fully declarative UI, ~30 screens, 291 composables
Concurrency	Kotlin Coroutines / Flow	Asynchronous repository calls and reactive UI state
Identity	Firebase Authentication	Email/password sign-in with OTP email verification
Data	Cloud Firestore	Real-time documents for users, posts, meals, chats, sleep
Messaging	Firebase Cloud Messaging	Push notifications for chat and social events

Layer	Technology	Role in Omni
AI	Google Gemini 2.5 Flash-Lite	Structured meal-nutrition estimation (server-side)
Backend	Node.js + Express	Privileged API: OTP, AI, verification, entitlement
Billing	RevenueCat 9.29	Omni+ subscription with server-side comp entitlement
Maps	MapLibre + MapTiler + OSRM	SOS emergency map, nearby hospitals, routing
Images	Coil 3.6 (+ GIF)	Async image loading and animated first-aid guides

Versions reflect the project's Gradle version catalogue and backend package manifest.

4 Methodology

4.1 Development Approach

The project followed an iterative and incremental development approach with Agile characteristics rather than a single-pass Waterfall model. Work was organized into sixteen numbered phases, each of which delivered a self-contained, testable increment of functionality — for example, an authentication phase, a social-feed phase, a nutrition phase, and a backend phase. Every phase concluded with verification on a physical Android device before the next began, so that regressions were caught early and each layer rested on a stable predecessor. This phase-based cadence provided the discipline of milestones while retaining the flexibility to reprioritize features as the design evolved.

Design fidelity was enforced continuously. The user interface was built as a one-to-one realization of a Figma design system, and screens were validated by side-by-side comparison of on-device screenshots against the Figma artboards, with programmatic measurement of element bounding boxes where pixel accuracy mattered. Version control was maintained with Git throughout.

4.2 Tools and Technologies

Development used Android Studio as the primary integrated development environment (IDE), with the Gradle build system driven by a Kotlin DSL and a centralized version catalogue. The client compiles against Android API level 37 and targets API level 35, with a minimum supported level of 26 (Android 8.0). Dependency injection is performed manually through a single application container rather than a framework such as Hilt or Dagger, keeping the object graph explicit and easy to follow. The backend was developed in TypeScript, run on Node.js 20, tested with the Vitest framework, and deployed to the Render cloud platform; Visual Studio Code was used for server development.

4.3 Project Timeline and Milestones

The table below summarizes the milestone structure. The phase numbers correspond to the increments actually delivered in the codebase; the grouping into milestones is presented here for clarity.

Table 2 Milestone structure and deliverables

Milestone	Phases	Key Deliverables
Foundation	0 – 2	Project scaffold, design system, theming, navigation shell
Core wellness	3 – 6	Home dashboard, step counter, sleep, nutrition logging
Content & social	7 – 9	Feed, stories, comments, likes, profiles, notifications
Care & safety	10 – 11	First-aid guides, SOS map, messaging, consultations
Backend & AI	12	Node/Express API, OTP verification, Gemini meal analysis
Monetization	13 – 14	Omni+ payroll, RevenueCat, server-side entitlement
Verification & polish	15 – 16	Doctor verification, motion polish, regression tests

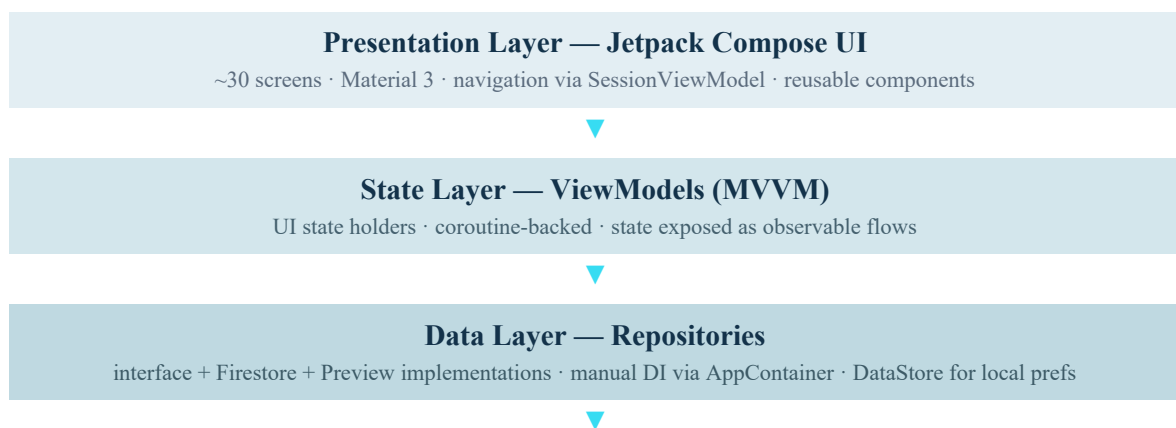
Each phase concluded with verification on a physical device before the next commenced.

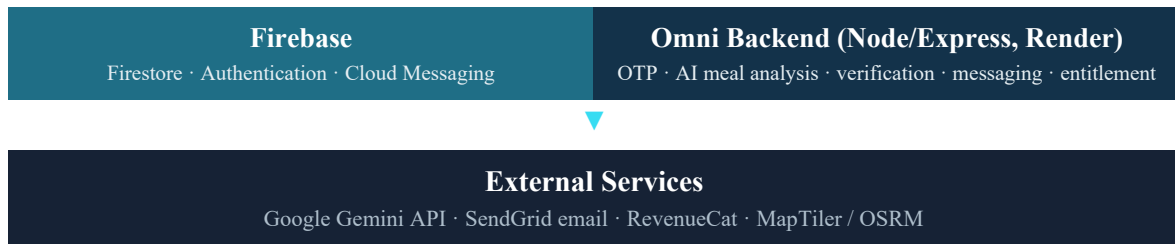
5 System Design and Architecture

5.1 High-Level Architecture

Omni adopts a layered client architecture that follows the Model-View-ViewModel (MVVM) pattern and communicates outward through two backends: Firebase for identity, data, and messaging, and a dedicated Node.js/Express service for privileged operations. The client never contacts the Gemini API, SendGrid, or the RevenueCat secret API directly; those flow exclusively through the Node service, which authenticates every request with a Firebase identity token. The figure below shows the flow of control from the user interface down to the external services.

Figure 1 High-level system architecture





All privileged calls (AI, email, secret-key entitlement) are brokered by the backend, never the client.

5.2 Core Components

The front end is the Compose UI, organized into feature packages (authentication, home, feed, messages, nutrition, sleep, first aid, SOS, Omni+, profile, settings, and notifications). Each feature exposes screens that observe a view model. The view models hold UI state and invoke repositories; they contain no Android framework or networking code directly, which keeps them unit-testable. Repositories are defined as interfaces with a real Firestore-backed implementation and a lightweight preview/fake implementation used for Compose previews and tests. Dependencies are wired together in a single application container that also acts as a view-model factory.

The backend is an Express application whose endpoints each perform one privileged task: sending and verifying a six-digit one-time password, analyzing a meal via Gemini, approving or rejecting doctor-verification requests, sending chat messages with a push notification, and toggling social likes and comments. A middleware verifies the caller's Firebase identity token on every request and derives the user identifier from the verified token rather than from the request body, closing a common spoofing vector. Firestore stores the application's documents — users, posts, meals, sleep records, conversations, doctors, and verification requests — and Cloud Messaging delivers notifications.

5.3 Data Model

The domain is expressed as a set of Kotlin data classes that map to Firestore documents, including User, Post, Story, Meal, MealAnalysis, SleepRecord, DailyMetrics, Conversation, Doctor, Consultation, VerificationRequest, Hospital, Route, EmergencyContact, Guide, and AppNotification. Counters such as follower and like totals are computed with server-side aggregation queries, and a like is modeled as one document per liker to keep writes idempotent and prevent double-counting.

5.4 UI/UX Design Considerations

The interface was implemented as a faithful reproduction of a Figma design system on a fixed reference artboard, giving the application a consistent visual identity across every screen. Reusable components — such as the verified-doctor badge and the Omni+ badge — were factored out and threaded through the app to guarantee consistency. Compose previews

follow a single approved convention (a five-pane device matrix wrapped in the application theme), so that every screen can be inspected without deploying to a device. Motion was treated as a first-class concern: a shared navigation-motion state lifts bottom-bar transitions above screen swaps so the navigation bar morphs smoothly instead of being recreated on each screen change. Accessibility and readability were maintained through Material 3 typography and sufficient color contrast.

6 Implementation

6.1 Development Process

Implementation proceeded feature by feature within each phase. For a typical feature, the team first built the Compose screen against a preview/fake repository so the layout could be validated against Figma, then implemented the real Firestore-backed repository and connected it through a view model, and finally verified the end-to-end behavior on a physical device. Sensitive operations were implemented last and on the server: only after a feature worked against Firestore was its privileged path (AI, email, or entitlement) routed through the Node backend. This ordering kept the client shippable at every step while ensuring secrets never reached it.

6.2 Key Features and Functionalities

The application delivers the following principal feature modules, each backed by its own view model and repository.

Table 3 Principal feature modules

Module	Functionality
Authentication	Email/password sign-up and sign-in with six-digit OTP email verification
Home dashboard	Bento-style aggregation of daily health metrics and updates
Step counter	Device step tracking with midnight-rollover handling, no foreground service
Sleep	Sleep logging, confirmation, and trend tracking
Nutrition	Manual meal logging plus AI meal analysis returning validated nutrition
Social feed	Posts, stories, comments, and likes with a followed-only story rail
Messaging	One-to-one Firestore conversations with backend send + push notification
First aid	Illustrated, animated step-by-step emergency guides (e.g., CPR)
SOS	MapLibre emergency map, nearby hospitals, and OSRM routing
Omni+	Subscription paywall, premium gating, doctor directory and consultation
Doctor verification	Clinician intake, admin approval, and entitlement comp

Each module follows the Screen → ViewModel → Repository → Firebase/backend flow.

6.3 AI Meal-Analysis Pipeline

The AI feature is representative of the security-first design and merits detailed description. When a user describes a meal, the nutrition view model calls a meal-analysis repository whose real implementation issues an authenticated HTTP request to the backend, attaching a freshly refreshed Firebase identity token and using a generous read timeout to tolerate cold starts on the hosting platform. The backend verifies the token, validates the input, and calls the Gemini generative-language model with a locked system prompt, a structured-output JSON schema, and a low temperature so that the response is deterministic and machine-parseable.

Crucially, the backend does not trust the model output verbatim. It re-validates the returned figures semantically — rejecting negative or implausible values and enforcing unit-aware quantity bounds — and it recomputes the meal totals server-side rather than accepting the model’s own arithmetic. Failures degrade gracefully to a manual-logging fallback, and the meal text is never written to logs. The Gemini API key exists only on the server, so no amount of client inspection can reveal it.

6.4 Code Structure and Major Modules

The Android source is organized under a single application package with clear top-level divisions: a data package (with model, repo, local, and revenuecat sub-packages), a dependency-injection package holding the application container, a domain package for recommendation logic, a service package for background work (cloud-messaging service, step-tracking service, and a boot receiver), and a ui package subdivided by feature. In total the client comprises approximately 40,000 lines of Kotlin across 151 source files and 291 composable functions. The backend is a compact fifteen-file TypeScript service in which each file owns one concern — for example, one module for Gemini access, one for meal analysis, one for messaging, one for verification, and one for RevenueCat entitlement — behind a shared authentication middleware and a Firestore-backed rate limiter.

7 Results and Discussion

7.1 Project Achievements

The project delivered a working, multi-feature Android application together with a deployed backend. All feature modules listed in Table 3 were implemented and verified on a physical device. The AI meal-analysis pipeline returns structured, server-validated nutrition data; the Omni+ subscription loads real offerings and prices and grants entitlements from the server; the verified-doctor consultation channel connects patients to clinicians; and the emergency

map renders nearby hospitals with routing. The interface matches the Figma design system to within a few device-independent pixels on the screens that were pixel-checked.

The system’s scale and test coverage are summarized below. These figures were obtained directly from the source tree and build configuration and are reported without embellishment.

Table 4 Selected project metrics

Metric	Value
Kotlin source files (client)	151 files, ~40,100 lines
Composable functions	291 across 52 files
Screens	~30
Backend TypeScript files	15 source + 6 test files
Client unit-test files	14 (JUnit4 + coroutines-test)
Minimum / target / compile SDK	26 / 35 / 37

Metrics reflect the state of the repository at the time of writing.

7.2 Challenges Faced and Solutions

Several substantive engineering problems were encountered and resolved during development. Each is described together with the solution adopted.

- **Protecting secrets** — The Gemini API key and RevenueCat secret could not live on the client. This was resolved by introducing the Node/Express backend and routing every privileged call through it, with the user identifier always taken from the verified Firebase token.
- **Untrusted AI output** — A language model can return malformed or implausible nutrition values. The backend enforces a strict output schema, semantically validates every field, and recomputes totals server-side rather than trusting the model.
- **Step-counter rollover** — Steps recorded before the app was opened were lost when the since-boot hardware counter re-anchored at midnight, causing mornings to read zero. This was fixed with an anchor-carry seed on same-boot day rollover plus a flush of the previous day’s tail, avoiding a battery-hungry foreground service.
- **Billing without a store** — Because subscription billing cannot be tested against a live store during development, the RevenueCat Test Store was used, and doctor entitlements are granted server-side by comping a fixed entitlement identifier that the code follows exactly.

- **Navigation motion** — The bottom navigation bar was being recreated on every screen change, breaking its morph animation. Lifting a shared navigation-motion state above the screen swap restored a continuous transition.

7.3 Comparison with Initial Objectives

Measured against the objectives stated in Section 2.2, the project met its goals. The table below records the outcome for each objective.

Table 5 Objectives versus outcomes

Objective	Outcome
Unified multi-feature client	Achieved — all modules implemented and device-verified
Security by design (no client secrets)	Achieved — all privileged logic behind the backend
AI-assisted, structured nutrition	Achieved — schema-constrained, server-validated Gemini pipeline
Tamper-resistant paid tier	Achieved — RevenueCat with server-side entitlement granting
Faithful realization of the Figma design	Achieved — pixel-checked against the design system
Public Play Store release	Not pursued — out of scope; distributed via sideloading

The final row was explicitly out of scope for the laboratory project.

8 Conclusion and Future Work

8.1 Summary of Findings

Omni demonstrates that a broad range of health-related concerns — activity, sleep, nutrition, first aid, social interaction, emergency assistance, and telemedicine — can be unified in a single, well-structured Android application without compromising security. The central architectural finding is that a thin, trusted backend is the right place for every privileged operation: AI inference, email verification, and subscription entitlement all belong on the server, where inputs can be validated, outputs can be re-checked, and secrets remain hidden. The MVVM-plus-repository structure on the client, combined with interface-based repositories and manual dependency injection, produced code that is testable and faithful to its design specification.

8.2 Project Limitations

The project has several honest limitations. The application uses a development package identifier and is distributed by sideloading rather than through the Play Store, so it is not yet a public release. Subscription flows were exercised against the RevenueCat Test Store rather than production billing. The step counter still has a gap when the device is rebooted before the application is next opened. Firestore security rules exist but are not yet covered by an automated rules-testing suite, and the backend’s deployment on a free hosting tier introduces occasional cold-start latency that the client mitigates with generous timeouts. Finally, the AI feature is limited to nutrition estimation and does not provide medical advice.

8.3 Future Work

Future development would proceed along several lines. The most immediate is a production release: migrating to a unique application identifier, adding a release signing configuration, and publishing to the Play Store. Beyond that, the team would add an automated Firestore rules-testing suite, move the backend to a warm hosting tier to eliminate cold starts, and close the reboot gap in the step counter. Feature-wise, promising directions include wearable-device integration for richer activity and heart-rate data, expansion of the AI pipeline to conversational health guidance with appropriate safeguards, richer analytics and trend visualizations on the home dashboard, and localization for additional languages. Together these steps would move Omni from a strong laboratory project toward a deployable consumer product.

In conclusion, Omni satisfies the objectives set for it: it is a unified, secure, and visually polished Android health application whose design decisions are defensible and whose implementation is grounded in a real, verifiable codebase.